

String Matching Algorithms

Pattern Matching is the way of comparing two or more text or array of characters.

It is also known as ~~to~~ string matching.

Its application can be seen in text editing program where a word supplied by user is matched in the given text using pattern matching process.

Naive Pattern Matching Algorithm Brute force Algorithm

Pattern matching algorithm generally follow two steps: Pre-processing of the string & then comparison with shifting of pattern (i.e. matching).

But in naive algorithm, no preprocessing is done.

Comparison is done left to right of characters in both the arrays.

If mismatch occurs, pattern is moved one position right & again comparison is done left to right.

Algorithm Naive String Matching
Inputs to algorithm are text T and
pattern to be matched, P .

Step 1 Find length of text
 $n \leftarrow \text{length}[T]$

Step 2 Find length of pattern
 $m \leftarrow \text{length}[P]$

Step 3 For every possible value of shift S
compare the characters in both the
arrays and print shift value on
Successful match.

For $S \leftarrow 0$ to $n-m$
if $P[1..m] = T[S+1..S+m]$ then
pattern matched & return S .

Step 4 Exit.

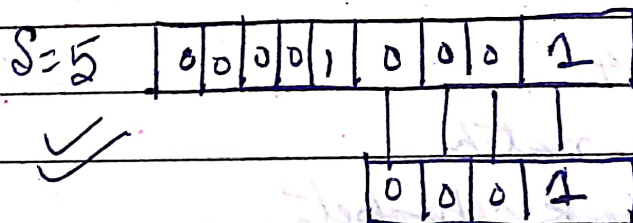
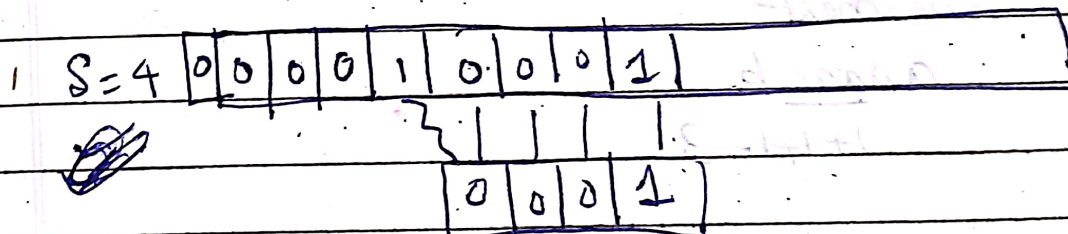
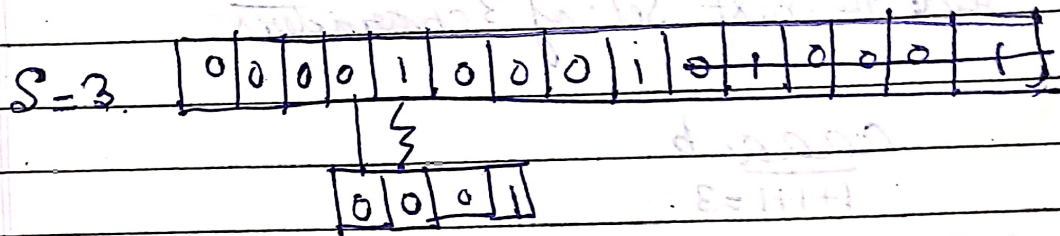
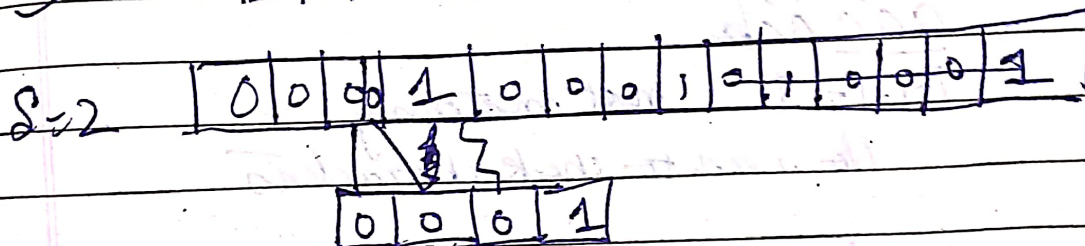
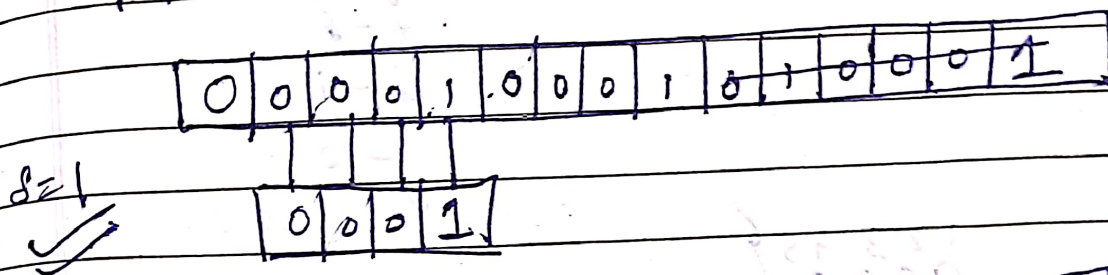
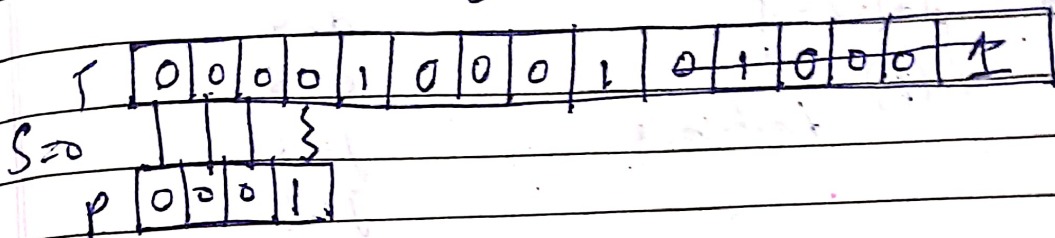
loop runs $(n-m+1)$ times
each time worst case m comparisons
So, complexity $O(m(n-m+1))$.

Example $P = 0001$

$T = 000010001010001$

$m = \text{length}(T) = 9$

$n = \text{length}(P) = 4$



0 to $n-m$
0 to $9-4$
0 to 5

Rabin Karp Algorithm

Text: ^{1 2 3 4 5 6}
aaaaab

n = 6

Pattern: ^{1 2 3}
aab

m = 3

$$1+1+2=4$$

$h(P)$

hash code

a-1

b-2

c-3

d-4

e-5

f-6

g-7

h-8

i-9

j-10

^{1 2 3 4 5 6}
aaaaab

$1+1+1=3$ not matching

No need to check characters

Slide to next set of 3 characters

aaaaab
 $1+1+1=3$

Slide next

aaaaab

$$1+1+1=3$$

aaaaab

$$1+1+2=4$$

we get a match

Now check alphabets

Matching

Pattern is found in $T[4..5]$

Text : $\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ a & b & c & d & a & b & c & e \end{matrix}$ $a-1$
 $n=8$

Pattern : $\begin{matrix} 1 & 2 & 3 \\ b & c & e \end{matrix}$ $b-2$
 $m=3$

$2+3+5=10$ hash value $c-3$

$d-4$

$e-5$

$f-6$

$g-7$

$h-8$

$i-9$

$j-10$

$\underline{abc}$
 $1+2+3=6$

$\underline{bcd}$ $6-1=5+4$

$2+3+4=9$

$\underline{cda} = 9-2+1=8$

$\underline{dab} \quad 8-3+2=7$

$\underline{abc} \quad 7-4+3=6$

$\underline{bce} \quad 6-5+5=10$

hash is matching

There is possibility of a pattern.

$b-b$

$c-c$

$c-e$

Avg. time of Rabin Karp Algorithm

$O(n-m+1)$

See the drawback of Robin Karp.

1 2 3 4 5 6 7 8 9 10 11
ccacc a a e d b a
 $3+3+1=7$

dba

$4+2+1=7$

simple hash fn. gives
collision with alphabets
that are matching called
as spurious hits.

a-1

b-2

c-3

d-4

e-5

f-6

g-7

h-8

i-9

j-10

$O(mn) \rightarrow$ worst case time

If no spurious hit $O(m-m+1)$ time.

Text: 1 2 3 4 5 6 7 8 9 10 11
ccacc a a e d b a
 $m=11$

pattern: 1 2 3
d b a

$m=3$

$$4 \times 10^2 + 2 \times 10^1 + 1 \times 10^0$$

$$P[1] \times 10^{m-1} + P[2] \times 10^{m-2} + P[3] \times 10^{m-3}$$

$$= 400 + 20 + 1 = 421$$

10 is base as we have taken 10 characters mapped.

cca

$$3 \times 10^2 + 3 \times 10^1 + 1 \times 10^0$$

$$300 + 30 + 1 = 331$$

Not a spurious hit now.

331

cac
 $3 \times 10^2 + 1 \times 10^1$

$(3 \times 10^2 + 1 \times 10^1) 10 +$

$\{ (3 \times 10^3 + 3 \times 10^2 + 1 \times 10^1) - 3 \times 10^2 \} \times 10 + 3 \times 10^0$

$= \frac{(331 - 300) 10 + 3}{3 \times 10 + 3}$
 $= \underline{313}$

$313 - 300 = 13 \times 10 + 3 \times 10^0$
 $= 130 + 3$
 $= \underline{133}$

$133 - 1 \times 10^2 = 33 \times 10$
 $= 330 + 1 \times 10^0$
 $= \underline{331}$

$331 - 3 \times 10^2 = 31$
 $31 \times 10 + 1 \times 10^0$
 $\underline{311}$

Reducing chance of spurious hits -

Boyer Moore algorithm

- Scans the characters of the pattern from right to left.
- If the text symbol that is compared with the rightmost pattern symbol does not occur in the pattern at all, then the pattern can be shifted by m positions behind this text symbol.

Boyer Moore is a combination of 2 approaches

- 1) Bad character Approach
- 2) Good Suffix Approach

Both of the approaches can also be used independently to search a pattern in a text.

Boyer Moore Algorithm does preprocessing to shift pattern by more than 1 character.

The character of the text that doesn't match the current character of the pattern is called Bad character.

We use the Bad-Match Table for shifting the pattern.

Algorithm

- 1) Construct bad-symbol shift table.
- 2) Align the pattern against beginning of the text.
- 3) Repeat the following step until either a matching substring is found or the pattern reaches beyond the last character of the text.

Text: "WELCOME TO TEAMMAST"

Pattern "TEAMMAST".

TEAM MAST

$$m = 8$$

$$\text{value} = \max\{1, m - \text{index} - 1\}$$

$$T = \max\{1, 8 - 0 - 1\} = 7$$

$$E = \max\{1, 8-1-1\} = 6$$

$$A = \max\{1, 8 - 2 \cdot 1\} = 5$$

$$M = \max\{1, 8-5-1\} = 4$$

$$\max \{1, 2-4-1\} = 3.$$

$$S = \max\{1, 2-6-1\} = 1$$

$$* = \rho = m$$

Set last character $T = \emptyset = m$

Bad character

WELCOM^{1 2 3 4 5 6}E TO TEAMMAST
TEAMMAST¹

See E in table, 6

move 6 characters forward, leaving W.

WELCOME TO TEAMMAST
TEAMMAST

$M=3$

WELCOME TO TEAMMAST
TEAMMAST

T = WELCOME TO JECRC

P = JECRC
0 1 2 3 4 m=5

$$J = \{5 - 0 - 1\} = 4$$

$$\therefore E = \{5 - 1 - 1\} = 3$$

$$C = 5$$

$$R = \{5 - 3 - 1\} = 1$$

$$* = 5 = m$$

Bad
1 2 3 4 5
WELCOME TO JECRC
JECRC

$$0 = * = 5$$

WELCOME TO JECRC
JECRC

$$J = 4$$

WELCOME TO JECRC
JECRC

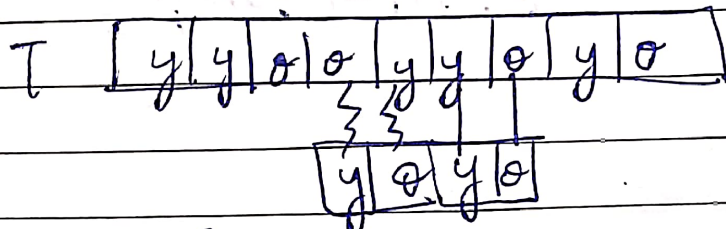
matched

Q Solve WELCOME TO JECRC COLLEGE
Pattern: — COLLEGE

Good Suffix Heuristic

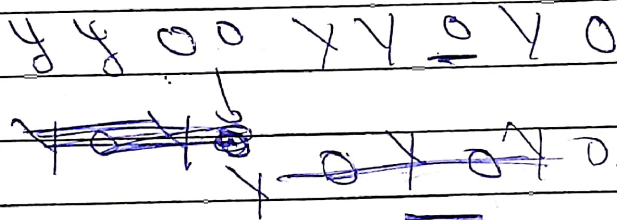
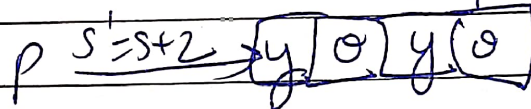
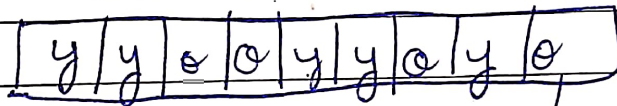
For a given pattern P , move R to L , if we have matched some suffix S , but there is mismatch with preceding character, then shift pattern in R direction, so that matched part is occupied by same suffix's.

If suffix S is nowhere in P , match largest prefix of P .



$S = 'yo'$

$S' = S + 2$



Pattern $P = \langle 2342 \rangle$

$T = \langle 21323422134561 \rangle$

2	1	3	2	3	4	2	2	1	3	4	5	6	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---

2	3	4	2
---	---	---	---

Shift by 3

2	3	4	2
---	---	---	---

2	3	4	2
---	---	---	---

shift

Compute Prefix Function (P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. for $q \leftarrow 2$ to m
5. do while $k > 0$ & $P[k+1] \neq P[q]$
6. do $k \leftarrow \pi[k]$
7. if $P[k+1] = P[q]$
8. do $k \leftarrow k+1$
9. $\pi[q] \leftarrow k$
10. Return π

$P: \boxed{a \mid b \mid a \mid b \mid a \mid c \mid a}$

$$m = 7$$

$$\pi[1] = 0$$

$$k = 0$$

Step 1: $q = 2, k = 0$
 $\pi[2] = 0$

q	1	2	3	4	5	6	7
P	a	b	a	b	a	c	a
π	0	0	1	2	3	0	1

Step 2 $q = 3, k = 0$
 $\pi[3] = 1$

Step 4 $q = 4, k = 1$
 $k = k+1 = 2$
 $\pi[4] = 2$

Step 5

$$q = 5, k = 2$$

$$k = k+1 = 3$$

$$\pi[5] = 3$$

Step 6

$$q = 6, k = 3$$

$$k = \pi[3] = 1$$

$$k = \pi[1] = 0$$

$$P[1] \neq P[6]$$

$$\pi[6] = 0$$

Step 7

$$q = 7, k = 0$$

$$k = 1$$

$$\pi[7] = 1$$

KMP Matcher (T, P)

$n \leftarrow \text{length}(T)$

$m \leftarrow \text{length}(P)$

$\pi \leftarrow \text{prefix fn}(P)$

$q \leftarrow 0$

$i = 1 \text{ to } n,$

do while $q \geq 0$ & $P[q+1] \neq T[i]$

$q \leftarrow \pi[q]$

if $P[q+1] = T[i]$

$q \leftarrow q + 1$

if $q = m$

{ then print pattern occurs with shift $i - m$

$q \leftarrow \pi[q]$ // loop for next match

}

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
T | b | a | c | b | a | b | a | b | a | b | a | c | a | c | a

1 2 3 4 5 6 7
P | a | b | a | b | a | c | a

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2	3	0	1

$$n = 15, m = 7$$

$$i = 1, q = 0$$

$$P[1] \neq T[1]$$

$$i = 2, q = 0$$

$$P[2] = T[2]$$

$$q = q + 1 = 1$$

$$i = 3, q = 1$$

$$P[3] \neq T[3]$$

$$q \leftarrow \pi[q]$$

$$q \leftarrow \pi[1]$$

$$q \leftarrow 0$$

$$i = 4, q = 0$$

$$T[4] \neq P[1]$$

$$q \leftarrow \pi[q]$$

$$i = 5, q = 0$$

$$a = a$$

$$q \leftarrow 1$$

$$i = 6, q = 1$$

$$b = b$$

$$q = q + 1 = 2$$

$$i = 7, q = 2$$

$$a = a$$

$$q = q + 1 = 3$$

$$i = 8, q = 3$$

$$b = b$$

$$q = q + 1 = 4$$

$$i = 9, q = 4$$

$$q = q + 1 = 5$$

$$i = 10, q = 5$$

$$q \leftarrow \pi[q]$$

$$q \leftarrow \pi[5]$$

$$q \leftarrow 3$$

$$i = 11, q = 3$$

$$q \leftarrow q + 1 = 4$$

$$i = 12, q = 4$$

$$q \leftarrow \pi[q] = 2$$

~~$$i = 13, q = 2$$~~

~~$$a = a$$~~

~~$$q = q + 1 = 3$$~~

~~$$i = 14, q = 3$$~~

~~$$q \leftarrow \pi[q]$$~~

~~$$q \leftarrow 1$$~~

~~$$i = 15, q = 1$$~~

~~$$b = b$$~~

~~$$q \leftarrow 4 (q + 1)$$~~

~~$$i = 11, q = 4$$~~

~~$$a = a$$~~

~~$$q \leftarrow 5$$~~

~~$$i = 12, q = 5$$~~

~~$$c, c$$~~

~~$$q \leftarrow 6$$~~

~~$$i = 13, q = 6$$~~

~~$$a, a$$~~

~~$$q \leftarrow 7$$~~

~~$$13 - 7 = 6$$~~

PAGE NO. :
DATE : / /

A flow network $G = (V, E)$ is a directed graph in which each edge $(u, v) \in E$ has a non-negative capacity $c(u, v) \geq 0$.

We distinguish two vertices in a flow network: a source s and sink t .

For convenience, we assume that each vertex lies on some path from source to sink.

For each vertex $v \in V$

flow network contains a path $s \rightarrow v \rightarrow t$.

The graph is therefore connected & since each vertex other than s has at least one entering edge, $|E| \geq |V| - 1$.

Let $G = (V, E)$ be a flow network with capacity function c .

$s \rightarrow$ source

$t \rightarrow$ sink

A flow in G is a real valued function $f: V \times V \rightarrow \mathbb{R}$ that satisfies the following 2 properties: -

Capacity Constraint: For all $u, v \in V$ we require $0 \leq f(u, v) \leq c(u, v)$.

Flow conservation: For all $u \in V - \{s, t\}$.

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$$

When $(u, v) \notin E$, there can be no flow from u to v , $f(u, v) = 0$.

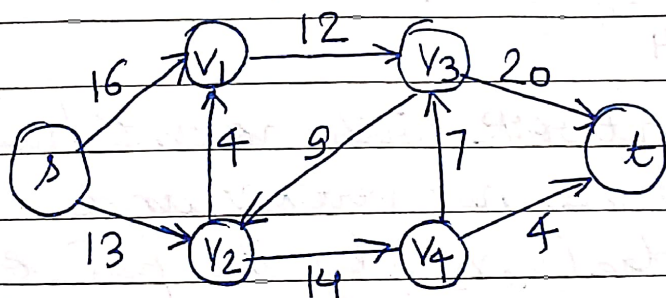
we call the non-negative quantity $f(u, v)$ the flow from vertex u to vertex v .

The value $|f|$ of a flow f is defined as

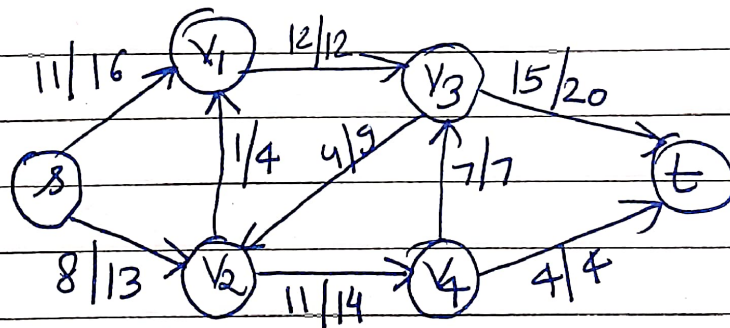
$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s).$$

i.e. Total flow out of the source - the flow into the source.

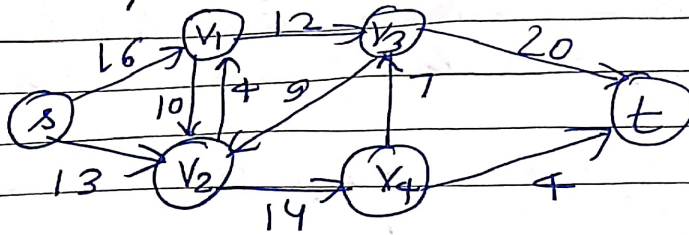
In the maximum flow problem, we are given a flow network G with source s and sink t , and we wish to find a flow of maximum value.



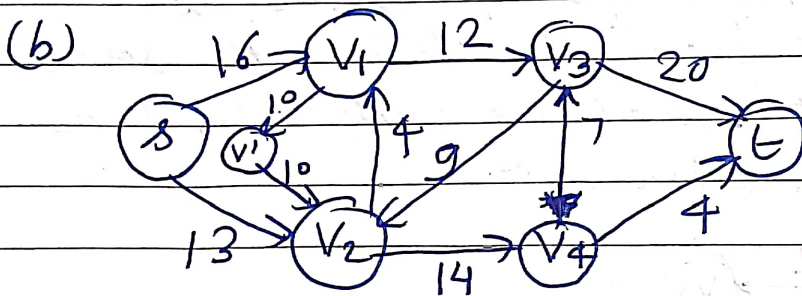
(a) flow network $G = (V, E)$



Anti-parallel edges.



(a) a flow network containing both the edges (v_1, v_2) and (v_2, v_1) .

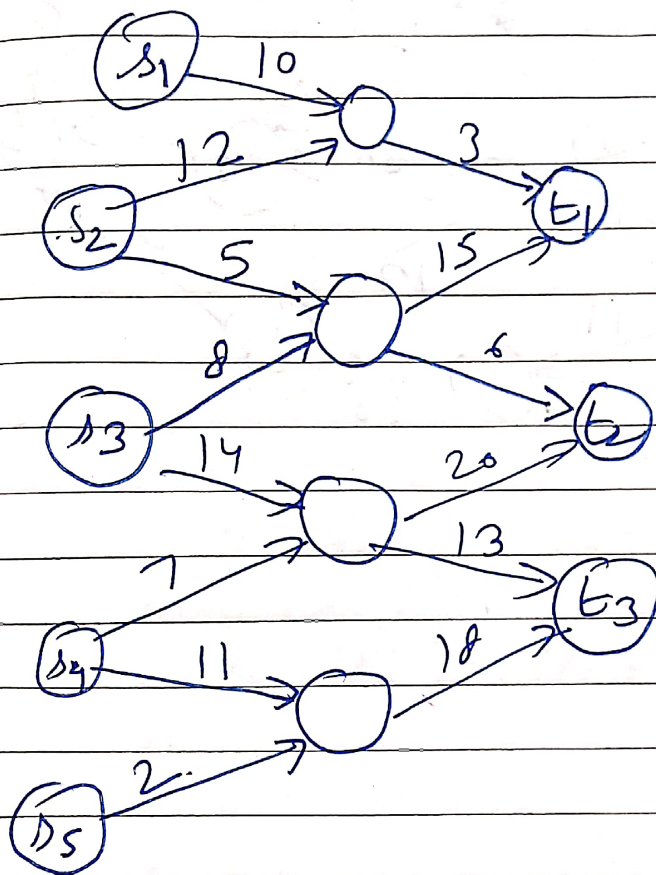


An equivalent network with no anti-parallel edge. We add the new vertex v' and we replace edge (v_1, v_2) by the pair of edges (v_1, v') and (v', v_2) both with the same capacity as (v_1, v_2) .

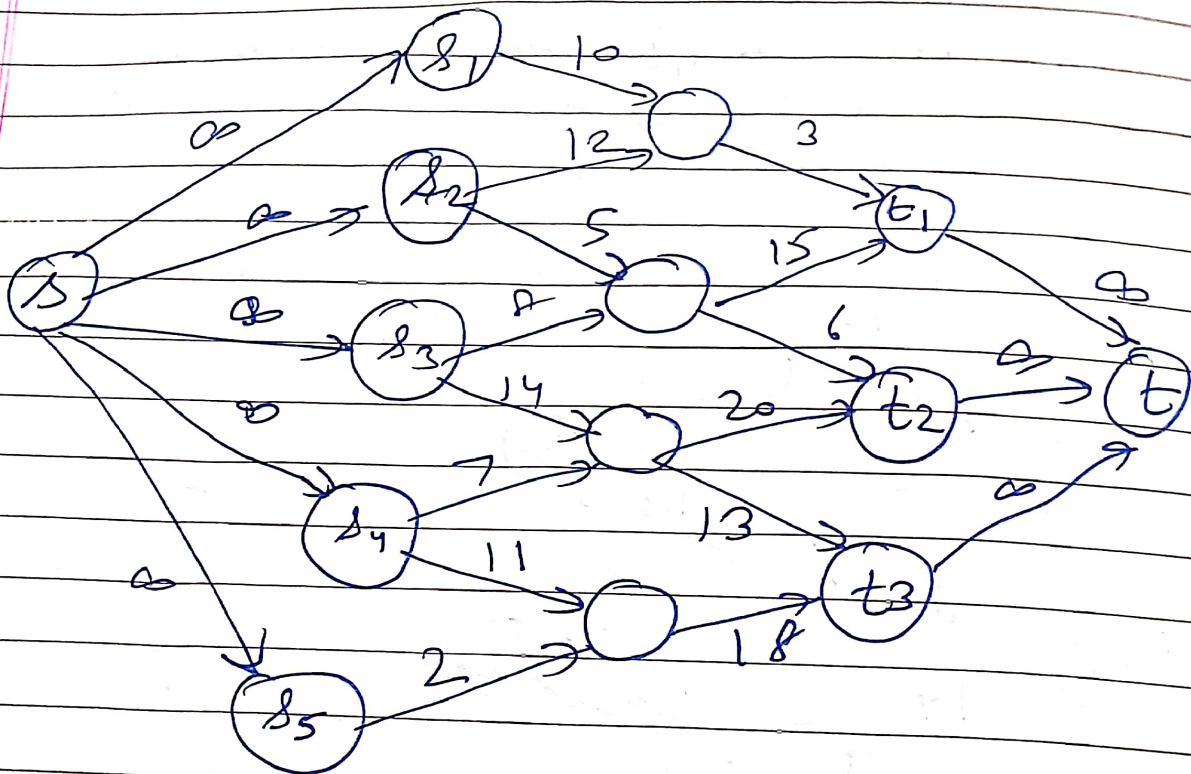
PAGE NO. :
DATE: / /

Multiple source, multiple-sink
maximum flow problem.

(A) A flow N/w with 5 sources
 $S = \{s_1, s_2, s_3, s_4, s_5\}$ and
3 sinks $T = \{t_1, t_2, t_3\}$.



(b) Equivalent single-source, single-sink flow network.



Ford-Fulkerson Method

we find an augmenting path in a residual n/w G_f and iteratively increase the flow.

Ford-Fulkerson Method (G, s, t)

1. initialize flow f to 0.
2. while there exists an augmenting path p in the residual network G_f .
3. augment flow f along p .
4. return f .

Given a flow N/w G and a flow f , the residual network G_f consists of edges with capacities that represent how we can change flow on edges of G .

An edge of the flow network can admit an amount of additional flow equal to edge's capacity - flow on that edge.

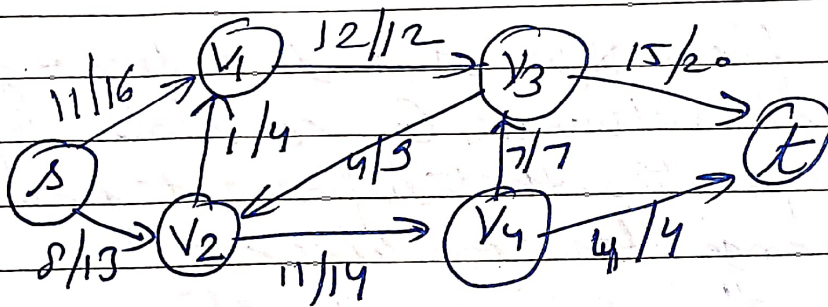
If that value is positive, we place that edge into G_f with residual capacity, $C_f(u, v) = c(u, v) - f(u, v)$.

The only edges of G that are in G_f are those that can admit more flow, those edges (u, v) whose flow equals their capacity have $C_f(u, v) = 0$ & they are not in G_f .

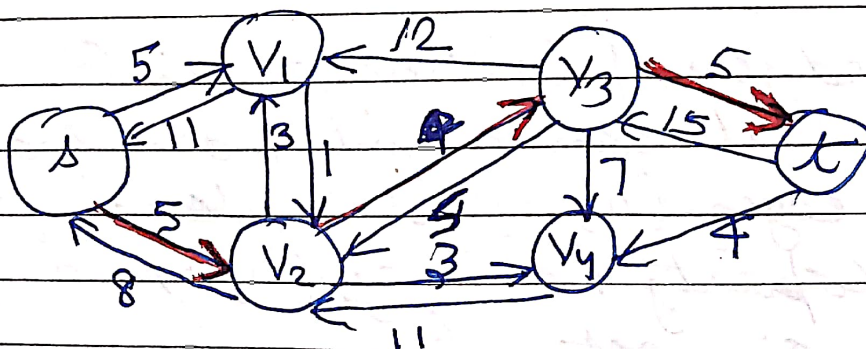
Suppose, we have a flow network $G = (V, E)$ with source s , and sink t .
 Let f be a flow in G , and consider a pair of vertices $u, v \in V$. We define the residual capacity $c_f(u, v)$ by

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{if } (u, v) \in E \\ f(v, u) & \text{if } (v, u) \in E \\ 0 & \text{otherwise} \end{cases}$$

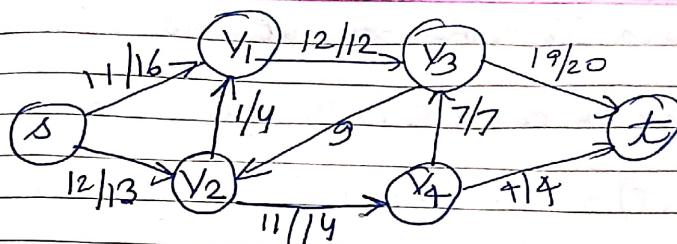
(a) flow network G and flow f



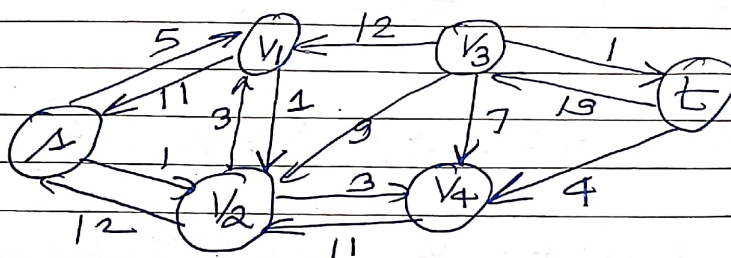
(b) residual network G_f with augmenting path p shaded, residual capacity $c_f(p) = c_f(v_2, v_3) = 4$.
 Edges with residual capacity 0 (v_1, v_3) not shown.



flow network
 s , and sink t .
 and consider
 v . We define
 $c_f(u, v)$ by
 $c_f(u, v)$ if $(u, v) \in E$
 $(v, u) \in E$
 else

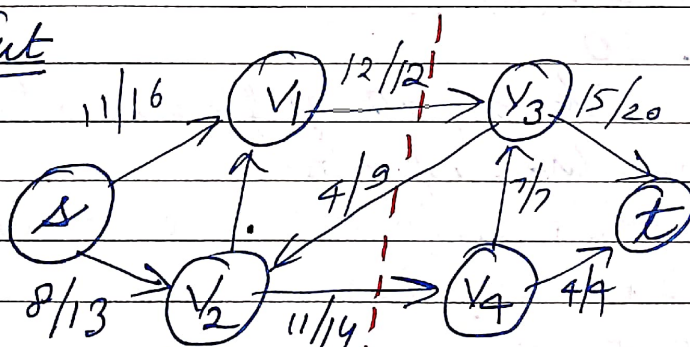


residual network.



Augmenting Path
 Given a flow network $G=(V, E)$ and flow f , an augmenting path p is a simple path from s to t in the residual N/w G_f .
 residual capacity of p ,
 $c_f(p) = \min \{ c_f(u, v) : (u, v) \text{ is on } p \}$

Cut



Page No. :
Date : / /

If f is a flow, then the net flow $f(S, T)$

across the cut (S, T) is defined as

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$$

Capacity of cut (S, T)

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$$

$$\text{cut}(\{s, v_1, v_2\}, \{v_3, v_4, t\})$$

$$f(v_1, v_3) + f(v_2, v_4) - f(v_3, v_2)$$

$$= 12 + 11 - 4$$

$$= 19.$$

Capacity of cut

$$c(v_1, v_3) + c(v_2, v_4) = 12 + 14 = 26.$$

Ford - Fulkerson Algorithm (G, s, t)

1. for each edge $(u, v) \in G, E$
 $(u, v) \cdot f = 0.$

while there exists a path p from s to t
in the residual n/w G_f .

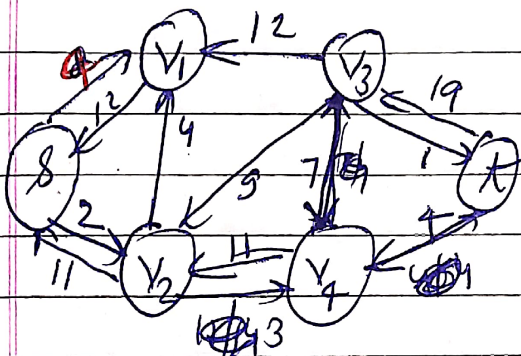
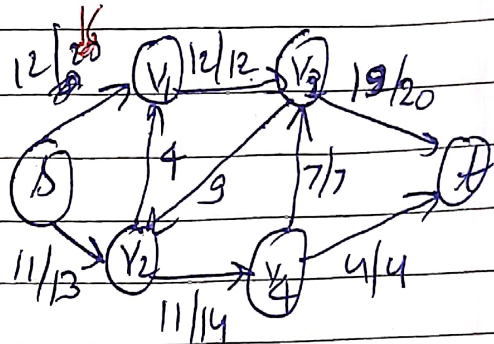
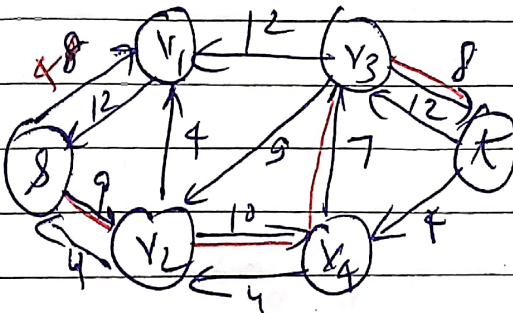
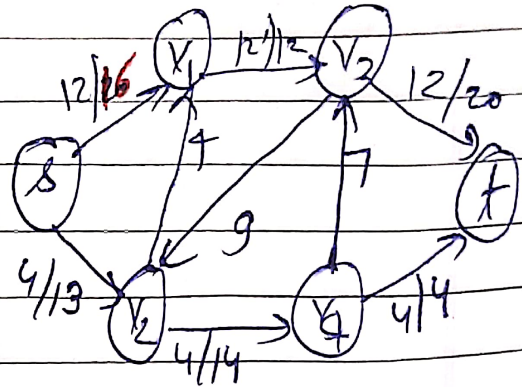
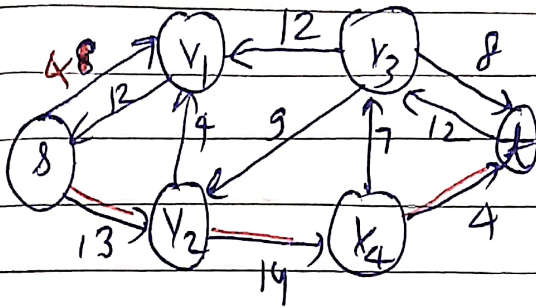
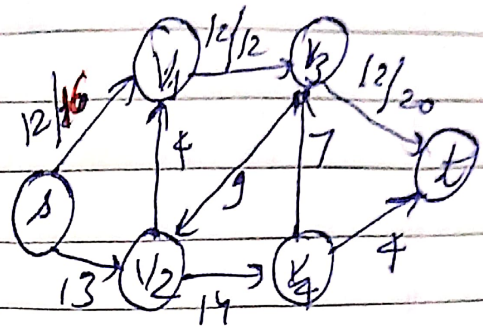
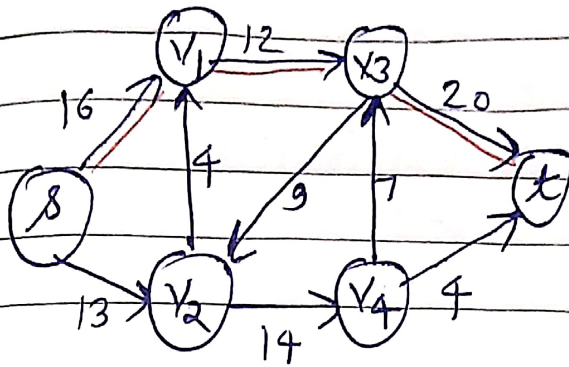
$$C_f(p) = \min \{ c_f(u, v) : (u, v) \text{ is in } p \}$$

for each edge (u, v) in p
if $(u, v) \in E$.

$$(u, v) \cdot f = (u, v) \cdot f + C_f(p)$$

$$\text{else } (v, u) \cdot f = (v, u) \cdot f - C_f(p)$$

Alternate Solution: -



$$12 + 11 = 23$$

$$19 + 4 = 23$$